

Problem ESM

Autor problemă:

Scurtă descriere

Se dă un șir de n numere întregi pozitive $v_1, v_2, \dots, v_n$ și un parametru $c \in \{1, 2, 3\}$. Problema solicită rezolvarea a trei cerințe distincte:

$c = 1$: Numărul de poziții i ($1 \leq i \leq n - 2$) pentru care $v_i \cdot v_{i+1} = v_{i+2}$ (progresie geometrică de trei termeni consecutivi).

$c = 2$: Se cere poziția j maximă astfel încât există $i < j$ cu $v_i \cdot v_j = v_n$ sau $v_i \mid v_n$ și $v_j = v_n/v_i$.

$c = 3$: Suma pozițiilor j maxime pentru fiecare poziție x , considerând același criteriu ca la $c = 2$ pentru v_x .

Soluție

Observăm că pentru cerințele $c = 2$ și $c = 3$, condiția $v_i \cdot v_j = v_n$ este echivalentă cu v_i fiind un divizor al lui v_n și $v_j = v_n/v_i$. Așadar, pentru fiecare element trebuie să găsim cel mai apropiat (prin poziție) divizor propriu care există în șir.

Observații cheie

Observație 1. Pentru a determina dacă un număr d apare în șir și care este cea mai recentă (respectiv a doua cea mai recentă) poziție, putem menține un tablou de frecvență $fr[x]$ care stochează ultima poziție unde a apărut valoarea x .

Observație 2. Similar, $fr2[x]$ poate stoca a doua cea mai recentă poziție a valorii x . După procesarea unui element la poziția pos , actualizarea este:

$$\begin{aligned} fr2[v_{pos}] &\leftarrow fr[v_{pos}] \\ fr[v_{pos}] &\leftarrow pos \end{aligned}$$

Observație 3. Numărul de divizori ai unui număr $v \leq 10^5$ este cel mult $O(\sqrt{v})$. Parcurgând doar până la $\sqrt{v}$, obținem toți divizorii în timp liniar în numărul de divizori.

Cazul $c = 1$

Verificarea este directă: parcurgem toate pozițiile i de la 1 la $n - 2$ și numărăm cazurile în care $v_i \cdot v_{i+1} = v_{i+2}$. Complexitate: $O(n)$.

Cazul $c = 2$

După citirea întregului șir, parcurgem toate elementele pentru a popula fr și $fr2$. Apoi, pentru fiecare divizor d al lui v_n , considerăm perechea $(d, v_n/d)$:

- Dacă $d \neq v_n/d$ și ambele valori apar în șir ($fr[d] > 0$ și $fr[v_n/d] > 0$), atunci poziția maximă validă pentru această pereche este $\min(fr[d], fr[v_n/d])$.
- Dacă $d = v_n/d$ (caz de pătrat perfect), avem nevoie de a doua cea mai recentă poziție, deci verificăm $fr2[d] > 0$.

Răspunsul este maximul dintre toate pozițiile valide. Complexitate: $O(\sqrt{v_n})$.

Cazul $c = 3$

Acest caz generalizează $c = 2$ prin aplicarea aceluiași raționament pentru fiecare element al șirului. Pentru fiecare poziție x , calculăm m_x (poziția maximă pentru v_x) și adăugăm m_x la răspuns. La final, actualizăm fr și $fr2$ pentru valoarea curentă.

Complexitate totală: $O(n \cdot \sqrt{\max v_i})$.

Implementare

```
int main() { ifstream cin("esm.in"); ofstream cout("esm.out");
    int c, n, v[100001], fr[100001] = 0, fr2[100001] = 0; cin >> c >> n;
    for(int i = 1; i <= n; i++) cin >> v[i];
    if(c == 1) int ans = 0; for(int i = 1; i + 2 <= n; i++) if(1LL * v[i] * v[i+1] == 1LL * v[i+2]) ans++;
    cout << ans << " ";
    if(c == 2) for(int i = 1; i < n; i++) fr2[v[i]] = fr[v[i]]; fr[v[i]] = i; int maxi = 0; for(int i = 1; i * i <= v[n]; i++) if(v[n] % i == 0) if(fr[i] > 0 && fr[v[n]/i] > 0) maxi = max(maxi, min(fr[i], fr[v[n]/i])); if(i == v[n]/i && fr2[i] > 0) maxi = max(maxi, fr2[i]);
    cout << maxi << " ";
    if(c == 3) long long ans = 0; for(int x = 1; x <= n; x++) int maxi = 0; for(int i = 1; i * i <= v[x]; i++) if(v[x] % i == 0) if(fr[i] > 0 && fr[v[x]/i] > 0) maxi = max(maxi, min(fr[i], fr[v[x]/i])); if(i == v[x]/i && fr2[i] > 0) maxi = max(maxi, fr2[i]); ans += maxi; fr2[v[x]] = fr[v[x]]; fr[v[x]] = x; cout << ans << " ";
    return 0; }
```

Explicație detaliată

Funcția de rezolvare a cazului $c = 2$ se bazează pe următoarea logică: pentru fiecare divizor i al lui v_n , obținem automat divizorul complementar v_n/i . Dacă ambele valori apar în șir la pozițiile p_1 și p_2 , atunci există o pereche validă cu poziția maximă $\min(p_1, p_2)$.

Pentru cazul $c = 3$, menținem același invariant: la pasul curent, $fr[x]$ reprezintă ultima poziție unde a apărut valoarea x , iar $fr2[x]$ reprezintă poziția anterioară. Astfel, când calculăm m_x , toate pozițiile din fr corespund doar elementelor din stânga lui x (deoarece nu am actualizat încă pentru v_x), ceea ce garantează corectitudinea.

Complexitate

- Cazul $c = 1$: $O(n)$ timp, $O(1)$ memorie adițională.
- Cazul $c = 2$: $O(n + \sqrt{v_n})$ timp, $O(\max v_i)$ memorie.
- Cazul $c = 3$: $O(n \cdot \sqrt{\max v_i})$ timp, $O(\max v_i)$ memorie.